Reinforcement Learning — Fall 2026

Homework 1: Introduction and Multi-armed Bandit

Theoretical and Implementation Parts

For any questions regarding this assignment, please contact
Amirparsa Bahrami at `amirparsa2828@gmail.com`.

Theoretical Part

1 Problem 1: Upper Confidence Bound (UCB) Action Selection

The Upper Confidence Bound (UCB) algorithm selects actions according to:

$$A_t = \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right]$$

Explain intuitively:

1. What each term represents.
2. Why the exploration bonus
$$\sqrt{\frac{\ln t}{N_t(a)}}$$
decreases with more samples.
3. How the constant c affects exploration.

2 Problem 2: Gradient Bandit Algorithm

In the gradient bandit algorithm, each action a is assigned a preference $H_t(a)$, and the agent chooses actions using a softmax distribution:

$$\pi_t(a) = \frac{e^{H_t(a)}}{\sum_b e^{H_t(b)}}$$

1. Starting from the objective
$$J(\mathbf{H}) = \mathbb{E}[R_t],$$
show that the gradient of the expected reward with respect to the preferences is:
$$\nabla_{H_t(a)} J = \mathbb{E}[R_t(\mathbb{I}_{a=A_t} - \pi_t(a))]$$
2. From this result, derive the stochastic gradient ascent update rule used in practice.
3. Explain intuitively how this update changes the action preferences when R_t is greater or less than the baseline $\bar{R}_t$.

3 Problem 3: Sample-Average and Constant Step-Size Updates

For a fixed action, let R_n denote the reward obtained on its nth selection, and let Q_n denote its value estimate immediately before that selection.

1. Starting from the sample-average estimate, show that its incremental update (for $n \geq 2$) is
$$Q_{n+1} = Q_n + \frac{1}{n}(R_n - Q_n).$$

2. Now consider a constant step size $0 < \alpha \leq 1$:
$$Q_{n+1} = Q_n + \alpha(R_n - Q_n).$$
By expanding this recursion, show that
$$Q_{n+1} = (1-\alpha)^n Q_1 + \sum_{i=1}^{n} \alpha(1-\alpha)^{n-i} R_i.$$

3. Using the weights in these two updates, explain which method is better suited to an environment where the mean rewards change over time. What trade-off arises when α is increased?

4 Problem 4: Optimistic Initial Action Values

Consider a *deterministic*, stationary three-armed bandit with rewards $r(1) = 1$, $r(2) = 2$, and $r(3) = 3$. The agent uses purely greedy action selection ($\varepsilon = 0$), updates selected actions using their sample-average rewards, and breaks ties by choosing the smallest action index.

1. Suppose the initial estimates are optimistic: $Q_1(a) = 5$ for every action a. Determine the first four selected actions and the value estimates after the third selection.

2. Repeat the first-four-actions calculation when $Q_1(a) = 0$ for every action. Explain why the behavior differs.

3. Explain how optimistic initialization can encourage exploration even with $\varepsilon = 0$. Why might this approach be insufficient when the true action values change over time?

Implementation Part

Question 1

Implement a 10-armed bandit problem. Assume each arm has a Gaussian reward distribution with the following parameters:

Mean: $[0.2, -0.7, 0.5, -0.3, 0.7, 0.9, 0.7, 0.6, 0.4, 0.3]$

Variance: $[0.2, 0.3, 0.6, 0.4, 1.0, 0.1, 0.7, 0.9, 0.1, 0.2]$

Arm Number	1	2	3	4	5	6	7	8	9	10
Mean	0.2	−0.7	0.5	−0.3	0.7	0.9	0.7	0.6	0.4	0.3
Variance	0.2	0.3	0.6	0.4	1.0	0.1	0.7	0.9	0.1	0.2

Part 1

(a) Implement the average-reward update rule:

$$Q_{k+1}(a) = Q_k(a) + \frac{1}{N_t(a)}\big(R_k(a) - Q_k(a)\big), \quad \forall a$$

Use an ε-greedy exploration algorithm with $\varepsilon = 0.1$. Initialize $Q(a) = 0$, and when multiple actions share the maximum Q value, select one of them at random.

Discuss what happens if we always select the greedy action (i.e., $\varepsilon = 0$).

Run this algorithm and plot the estimated Q_k values over time, marking the chosen arm as in Fig. 1, for the first 20 steps and for 1000 steps.

time step	1	2	3	4	5	6
arm 1	0.2	0.3	0.3	0.3	0.3	0.3
arm 2	0.4	0.4	0.4	0.4	0.4	0.4
arm 3	0.7	0.7	0.7	0.7	0.7	0.7
arm 4	0.2	0.2	0.2	0.2	0.2	0.2
arm 5	0.1	0.1	0.5	0.5	0.5	0.5
arm 6	0.3	0.3	0.3	0.3	0.3	0.8
arm 7	0.6	0.6	0.6	0.6	0.6	0.6
arm 8	0.8	0.8	0.8	0.8	0.8	0.8
arm 9	1	1	1	0.2	0.2	0.2
arm 10	0.6	0.6	0.6	0.6	0.1	0.1

Figure 1: Estimated reward of each arm vs. time.

(b) Plot the reward at each time step, and also plot the windowed average reward on the same figure using a different color, with window_size = 10.

(c) Repeat the experiment $N_{\text{runs}} = 2000$ times and plot the average reward versus time step and the percentage of optimal action versus time step. (All averages and percentages should be computed across different runs.)

From this point onward, we analyze the performance of each algorithm independently of the specific reward values of the arms.

To ensure this independence, all experiments are repeated $N_{\text{runs}} = 2000$ times.

In each independent run, the true mean reward of arm i, denoted by $q_*(i)$, is randomly drawn from a normal distribution:

$$q_*(i) \sim \mathcal{N}(0, 1)$$

For each arm i, the true mean of the reward is $q_*(i)$, and the variance is fixed to 1. Consequently, the reward distribution at time step t follows:

$$R_t \sim \mathcal{N}\big(q_*(A_t),\, 1\big)$$

(d) Plot the average reward versus time step and the percentage of optimal action versus time step. (All averages and percentages should be computed across different runs.)

Part 2

Implement the constant step-size update rule:

$$Q_{t+1}(A_t) = Q_t(A_t) + \alpha\big(R_t - Q_t(A_t)\big)$$

(a) Plot the average reward versus time step and the percentage of optimal actions versus time step for

$$\alpha \in \{0.05,\, 0.1,\, 0.2,\, 0.4,\, 0.8,\, 1.6\}.$$

Determine which value of α yields the best performance. What happens if α is too low or too high?

(b) Plot the results for the best α—including both the average reward and the percentage of selecting the best arm—in the same graph as the results obtained in Part 1(d) (i.e., the plot for the average-update rule).
Evaluate and compare the time required to reach an 80% probability of selecting the optimal action for both the constant step-size and average-reward algorithms.
Determine which method converges faster to the best arm.

Part 3

Now test the algorithms in a **non-stationary** environment. In this setting, the mean reward of each arm changes randomly at every time step, following a random walk process with variance 0.1. That is, for each arm i,

$$q_*(i) \leftarrow q_*(i) + \mathcal{N}(0,\, 0.1^2)$$

Run experiments using:

1. The average-reward update method, and
2. The constant step-size method with $\alpha \in \{0.05,\, 0.1,\, 0.2,\, 0.4\}$.

Plot the average reward and the percentage of selecting the best arm over time. Compare the performance of these two algorithms in the non-stationary environment.

Question 2: The 10-Armed Bandit Problem

In this assignment you will reproduce and extend several classical experiments from Chapter 2 of Sutton and Barto's *Reinforcement Learning: An Introduction*. The goal is to compare how different action-selection and value-update strategies behave in multi-armed bandit.

General Setup

- Number of arms (actions): $k = 10$.
- For each arm i:
$$q_*(i) \sim \mathcal{N}(0, 1)$$
- Reward distribution:
$$R_t \sim \mathcal{N}(q_*(A_t), 1)$$
- Simulation parameters:
  - Steps per run: $T = 500$,
  - Independent runs for averaging: $N_{\text{runs}} = 2000$.

Initialize $Q(a) = 0$ and $N(a) = 0$ for all a, and break ties randomly among equally valued actions.

Task 1 — The ε-Greedy Algorithm

Goal: Investigate how the exploration parameter ε affects learning.

- Implement the ε-greedy policy:
$$A_t = \begin{cases} \text{random action}, & \text{with probability } \varepsilon, \\ \arg\max_a Q_t(a), & \text{with probability } 1 - \varepsilon. \end{cases}$$
- Use the sample-average update for $Q_t(a)$.
- Evaluate $\varepsilon \in \{0, 0.01, 0.1, 0.3, 0.6, 0.9\}$ in the stationary environment.
- **Plots:** average reward and % optimal action vs. time step.
- **Discussion:** Explain the short-term and long-term impact of increasing ε. Which ε performs best overall, and why?

Task 2 — UCB vs. ε-Greedy: Regret Comparison

Goal: Compare the performance of UCB and ε-greedy in terms of cumulative regret.

- Implement the Upper Confidence Bound (UCB) rule:
$$A_t = \arg\max_a \Big[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \Big],$$
where c controls exploration intensity.
- Use $c \in \{1, 2, 5\}$ and the sample-average update.

- Define instantaneous regret $\Delta_t = q_*(a^*) - q_*(A_t)$ and cumulative regret

$$R(T) = \sum_{t=1}^{T} \Delta_t, \quad a^* = \arg\max_a q_*(a).$$

- Compare UCB (different c) with ε-greedy ($\varepsilon = 0.1$).
- **Plots to produce:**
  1. Average reward vs. time step,
  2. Percentage of optimal action vs. time step,
  3. Cumulative regret $R(T)$ vs. time step (log scale recommended).
- **Discussion:**
  - How does c influence exploration?
  - Does UCB achieve lower regret than ε-greedy?
  - Describe early-time and asymptotic behavior of both methods.

Task 3 — Gradient Bandit and Baseline Analysis

Goal: Study the effect of including a baseline on the gradient bandit method.

- Implement the softmax policy:

$$\pi_t(a) = \frac{e^{H_t(a)}}{\sum_b e^{H_t(b)}}.$$

- Update preferences as:

$$H_{t+1}(a) = H_t(a) + \alpha\,(R_t - b)\,\big(\mathbb{I}_{a=A_t} - \pi_t(a)\big), \quad \forall a.$$

  where b_t is either:
  - the average reward $\bar{R}_t$ (**with baseline**), or
  - 0 (**without baseline**).
- For this task, generate the true mean rewards of the arms from

$$q_*(i) \sim \mathcal{N}(5, 1).$$

- Test $\alpha \in \{0.1, 0.4\}$ in the stationary setting.
- **Plots:** average reward and % optimal action vs. time step.
- **Discussion:**
  - How does the baseline affect learning stability and variance?
  - How does α influence convergence speed and smoothness?